



Dúvida CKP1 - Igor Roberto

2 mensagens

Igor Roberto Michalski de Souza <rgm47539@comp.uems.br>
Para: Rubens Barbosa Filho <rubens@comp.uems.br>

qua., 16 de set. de 2026 às 16:24

Boa tarde Professor. Tudo bem?

Não consegui estar presente na sua aula hoje então pedi para o Thainan te perguntar sobre a questão do Ping/Pong que tem no seu script de teste que foi enviado para o Kauan.

Thainan me passou que esse Ping ele funcionaria somente para ver se as máquinas estão conectadas e que não seria necessário implementar eles como uma instrução como LEAVE, JOIN, etc.

Olhando o script de teste, o ping é passado como argumento para o nosso executável:

```
"$CLIENT" --cmd ping --host 127.0.0.1 --port "$PORT" >/tmp/c1_ping.out  
2>/tmp/c1_ping.err
```

Pode me confirmar como devo tratar esse ponto ou me dizer se há uma forma mais adequada para ser feito isso.

Atenciosamente,
Igor Roberto Michalski de Souza
Ciência da Computação

Rubens Barbosa Filho <rubens@comp.uems.br>
Para: Igor Roberto Michalski de Souza <rgm47539@comp.uems.br>

qui., 17 de set. de 2026 às 01:35

Olá Igor.

Em relação ao comando Ping, quando o Thainan perguntou, eu entendi que se tratava do ping que usamos para pingar uma máquina e não do ping pong. Mas vamos às suas dúvidas.

O ping/pong precisa ser sim implementado, mas é algo bem trivial.

Eu uso a função `getopt_long` do C.

Basicamente o que eu faço é executar a função `getopt_long` a cada iteração. A função identifica qual opção foi passada (ex: -c ou --cmd), retorna o caractere dessa opção e o guarda em uma variável.

Se a opção possuir um valor (como -p 5101), o texto 5101 é armazenado automaticamente na variável global.

No caso do ping, eu uso uma string de opções curtas que aceita (com um hífen só, ex: -c). O caractere de dois-pontos (:) logo após cada letra significa que **aquela opção obrigatoriamente exige um argumento extra** (ex: -c ping ou -p 5101).

Sendo mais preciso, o MSG_PING é um identificador (um valor de enum) que eu chamo do protocol.h (esse protocol.h) está listado no checkpoint 1. Eu fiz ele em três etapas:

1) No main, eu passo o argumento -c ping na linha de comando, a função main detecta a string e chama uma função repassando a constante MSG_PING e a payload "PING":

```
if(!strcmp(c,"ping")) return simple(h,p,MSG_PING,"PING"); Esse é literalmente o meu código.
```

2) Nos itens de verificação do checkpoint 1 tem a interpretação do header. Aqui eu faço o empacotamento. Basicamente, eu uso uma função tipo simple recebe MSG_PING no parâmetro MessageType t e o atribui ao campo type do cabeçalho da mensagem (MessageHeader q). No meu código fica assim:

```
MessageHeader q={0};  
q.version = 1;  
q.type = t; // Aqui q.type recebe MSG_PING  
q.timestamp = (uint64_t)time(NULL);  
q.payload_size = (uint32_t)strlen(s); // Tamanho de "PING" (4 bytes)
```

3) Por fim, eu envio a resposta. O cabeçalho preenchido junto com a string "PING" são enviados para o servidor de destino via `protocol_send`. O código converte o tipo numérico em texto chamando `message_type_name(t)` para imprimir no terminal (TX PING). O programa aguarda a resposta do servidor com `protocol_recv` e imprime o resultado antes de fechar a conexão.

O PONG é a mesma coisa.

Sua última dúvida é em relação à passagem de argumento pelo script. Também é tranquilo. Você precisa gerar os executáveis. Depois você chama o script passando algumas flags tipo -c, -h, -p como faria normalmente usando o terminal.

Exemplo: `./script.sh -c ping -h 192.168.1.100 -p 5101`

Mais especificamente: `./meu_cliente --cmd upload --host "$HOST_PADRAO" --port "$PORTA_PADRAO"`

Um detalhe importante: Como a função `getopt_long` no C lê o vetor `argv` do processo criado pelo Bash, o programa em C processará as flags da mesma forma, independentemente de terem sido digitadas manualmente ou montadas pelo script.

Aproveitando:

O arquivo `network.c` trata das funções `listen` e `connect`. O arquivo `protocol` é a formatação (sintaxe) de comunicação. O `peer.c` trata do `NodelInfo` e das funções de `upload` e `download` (essas serão utilizadas no checkpoint 2). O `node.c` é o servidor. E o `superpeer.c` vai tratar do `chord` que veremos bem mais a frente.

Aproveitando mais ainda. Não sei se vocês conseguiram entender meu script, mas segue abaixo uma explicação mais detalhada: A ideia é validar a implementação da camada de rede (servidor TCP, protocolo, framing de mensagens e concorrência).

Configuração do Ambiente e Variáveis

- **set -u**: Força o Bash a emitir um erro e interromper a execução caso tente utilizar uma variável não declarada.
- **ROOT, C1DIR, NODE, CLIENT**: Mapeia a estrutura de diretórios e localiza os binários do servidor (bin/node) e do cliente (bin/client).
- **PORT, NAME, LOG**: Define a porta TCP (padrão 55101), o nome do nó e o arquivo onde a saída do servidor será salva.
- **pass e fail**: Contadores globais do resultado dos testes.

Funções Auxiliares e Limpeza Automática

- **ok() / notok()**: Formata o resultado no terminal em cores/texto ([PASS] ou [FAIL]) e atualizam os contadores.
- **cleanup() + trap**: Garante que o processo do servidor (NODE_PID) rodando em segundo plano seja encerrado via kill e wait quando o script terminar ou for interrompido (Ctrl+C, erro ou encerramento).

Validação de Compilação e Testes Unitários

- Verifica se os executáveis bin/node e bin/client existem.
- Compila a pasta de testes com make e executa a suíte de testes unitários do protocolo (test_protocol).

Inicialização do Servidor e Polling de Prontidão

- Limpa logs antigos e executa o servidor node em segundo plano (&), salvando seu PID na variável NODE_PID.
- Faz um laço de até 30 tentativas (com pausa de 0,1s) enviando um --cmd ping via cliente C até que o servidor aceite conexões TCP.

Testes de Comandos Individuais

- **PING → PONG**: Valida se a resposta salva na checagem inicial contém a string RX PONG.
- **JOIN → ACK**: Executa o comando join no cliente e verifica se a resposta do servidor foi RX ACK.
- **LEAVE → ACK**: Envia a mensagem de saída leave e checa a confirmação RX ACK.

Teste de Carga e Concorrência

- **20 Clientes Concorrentes**: Dispara 20 processos do cliente simultaneamente em segundo plano com &, aguarda todos terminarem (wait) e confirma se todos os 20 receberam a resposta RX PONG.
- **Framing no Servidor**: Abre o arquivo de log do servidor e conta se há pelo menos 21 registros de RX PING recebidos (1 de prontidão + 20 concorrentes).

Injeção de Pacote Inválido e Sanidade de Log

- **Script Python Inline**: Abre um socket bruto e injeta um pacote TCP com versão de protocolo inválida (version=99). O teste passa se o servidor rejeitar a conexão sem responder dados (comportamento de segurança esperado).
- **Sanidade do Log**: Confirma se o nó registrou corretamente seu nome e ID durante a inicialização.

Relatório Final

Soma a quantidade de testes aprovados e reprovados. Se fail for igual a 0, encerra com status 0 (aprovado); caso contrário, mostra o caminho do log e encerra com status 1 (reprovado).

É isso. Compartilha com os demais colegas. Pode ser útil para eles.
Qualquer dúvida é só falar.
fallows.

Prof. Dr. Rubens Barbosa Filho
Curso de Ciência da Computação
Universidade Estadual de Mato Grosso do Sul - UEMS.
Dourados - MS.

[Texto das mensagens anteriores oculto]